

Task Plan — Central da Nação

Projeto: hub informativo só da torcida do Flamengo (Agora / Nação / Cortes / Perto de você).

Protocolo: B.L.A.S.T. (Blueprint, Link, Architect, Stylize, Trigger) + arquitetura A.N.I. (Architecture / Navigation / Instruments-Tools).

Origem: blueprint `BLAST_Central_da_Nacao_SYSTEM_PROMPT.md` v1.0 (2026-09-09) — a seção **BLUE-PRINT** abaixo está copiada integralmente, conforme Protocol 0.

Regra de avanço: uma fase só abre depois que os checklists da fase anterior estão verdes. Halt Execution permanece válido até a confirmação humana para a Phase L (Link).

BLUEPRINT (cópia integral da Phase 1 — B)

1. North Star

Único resultado desejado:

Todo dia, sem falha, a Central da Nação publica um **Daily Payload** completo e um **Match Payload** (quando houver jogo), no destino final na nuvem, para que o torcedor abra um único lugar e saiba: o que aconteceu, o que a Nação está falando, o que rir, e onde ir na cidade dele.

Métrica de “Complete” (regra BLAST): o projeto SÓ está completo quando o payload está no destino cloud (não quando o scrape rodou no `.tmp/`).

Não é North Star:

- virar app nativo
- transmitir o jogo com imagens oficiais
- substituir Fla-APP / Flamengo TV
- ser mais um portal que republica matéria inteira

2. Integrations

Usar o mínimo no MVP. Keys podem NÃO estar prontas — Phase L testa e registra o que falta em `findings.md`. Nunca inventar credencial.

Serviço	Uso	Obrigatório no MVP	Key
API-Futebol (api-futebol.com.br) ou API-Football (api-sports.io)	jogos, placar, escalação, tabela	SIM (uma das duas)	API_FUTEBOL_KEY
RSS / HTTP das fontes da allowlist	notícias	SIM	nenhuma
OpenAI ou Anthropic ou Gemini	resumir card, classificar, deduplicar	SIM	LLM_API_KEY
Google Sheets ou Supabase	source of truth + payload final	SIM (escolher um; default Sheets)	GOOGLE_SERVICE_ACCOUNT ou SUPABASE_URL + SUPABASE_KEY
Slack ou Discord webhook	fila de aprovação editorial + alertas de erro	SIM	EDITOR_WEBHOOK_URL
Resend ou Gmail API	digest e-mail (fase 2)	NÃO no MVP	RESEND_API_KEY
WhatsApp Cloud API / Z-API	alerta gol/escalação (fase 2)	NÃO no MVP	—
X API	timeline (fase 2)	NÃO no MVP	cara; usar allowlist + embed
YouTube Data API	cortes publicados por nós	fase 2	YOUTUBE_API_KEY
Google Maps Geocoding	lat/Ing de bares	fase 2	GOOGLE_MAPS_KEY

Default MVP de persistência: **Google Sheets** (3 abas: `news_cards` , `matches` , `venues`) + webhook Slack/Discord para o editor.

Se a key de futebol não existir, o Pilot constrói o handshake com fixture mock e deixa o adapter real atrás de uma interface. Não bloqueia o resto.

3. Source of Truth

Dado	Onde vive	Quem escreve
Fontes de notícia allowlist	geminimd + aba sources	humano
Contas X allowlist	geminimd + aba accounts	humano
Cards de notícia processados	Sheets/DB news_cards	tools + editor
Jogos / placar / escalação	API de futebol → matches	tools
Bares / lojas / embaixadas	aba venues	humano no dia 1, depois o estabelecimento
Regras editoriais / tom	geminimd	humano
Fila de aprovação	Slack/Discord	tools postam, humano reage ✅✍️🗑️
Payload do dia publicado	Sheets aba daily_payload + (fase S) página HTML	tools após aprovação

Nunca tratar .tmp/ como verdade. .tmp/ é bancada. Verdade = cloud.

4. Delivery Payload

Destinos, em ordem de prioridade:

1. **Google Sheets / Supabase** — registro canônico do dia (daily_payload JSON)
2. **Webhook Slack/Discord** — bloco formatado “Hoje na Nação” + fila de aprovação
3. **Arquivo HTML estático** em cloud (Cloudflare Pages / GitHub Pages / bucket) — a “home” pública do MVP
4. Fase 2: e-mail digest 07:30 BRT e push WhatsApp

O payload do dia tem 4 seções obrigatórias: agora , nacao , cortes , perto_de_voce .

Se uma seção estiver vazia, o campo existe com items: [] e status: "empty" — nunca omitir a chave.

5. Behavioral Rules

Tom

- Português BR, 2ª pessoa ou “a Nação”.
- Direto. Frases curtas. Sem “conforme divulgado pelo departamento de comunicação”.
- Pode ser irônico. Não pode ser xenófobo, machista, homofóbico, ou atacar jogador da base menor de idade.
- Rival: provoca o clube, não a torcida rival como gente.
- Nunca fale como se fosse o Clube de Regatas do Flamengo. Assine sempre como Central da Nação / veículo independente.

Regras de lógica (Do / Do Not)

DO:

- Sempre citar a fonte original no card. Link para a origem. Não republicar o texto integral.
- Selo obrigatório: `oficial` | `confirmado` | `rumor` | `opinioao` | `meme` | `ugc`.
- `oficial` SÓ se a URL for do domínio allowlist oficial (flamengo.com.br, redes oficiais).
- `rumor` se a fonte não for oficial E o texto contiver “negociando / perto / sondado / exclusividade” sem confirmação cruzada.
- Deduplicar notícias com similaridade de título+entidades > 0.82. Manter a mais antiga + a oficial se existir.
- Horário oficial do sistema: **America/Sao_Paulo**.
- Em dia de jogo do profissional masculino: priorizar match center no topo do `agora`.
- Cortes de LANCE: somente UGC com crédito OU material próprio. Nunca vídeo da FlaTV / Globo / ESPN para monetizar.
- Meme: só da allowlist de contas + UGC enviado. Descartar deepfake de jogador e conteúdo sexual com menor.
- Venue: só aparece no payload do dia se `status=verified` e `city` bater com a cidade do usuário/default.

DO NOT:

- Não inventar escalação, rumor ou placar.
- Não usar marca/escudo oficial como logo do produto.
- Não monetizar conteúdo oficial do clube.
- Não postar automaticamente na home pública sem passar pela fila (MVP). Flag `AUTO_PUBLISH=false` até o editor mandar o contrário.
- Não chamar API do X no MVP.
- Não scrapar atrás de paywall.
- Não adivinhar business logic. Se o selo ficar ambíguo → `needs_review: true`.

Human-in-the-loop (MVP)

- 07:15 BRT: pipeline monta rascunho → webhook
- Editor tem 30 min para  /  / 
- 07:45: se não houver resposta E `AUTO_PUBLISH=false`, o payload fica em `status=draft` (não vai pra home)
- Em jogo: pipeline de evento (gol, intervalo, fim) pode publicar match ticks sem aprovação, porque vêm da API, não da IA

Fases e checklists

Protocol 0 — Initialization

Objetivo: memória do projeto pronta e Halt Execution liberado para a Phase L (somente com OK humano).

- [x] `task_plan.md` criado (este arquivo) com o Blueprint integral
- [x] `findings.md` criado (pesquisa GitHub + constraints + lacunas de credenciais)
- [x] `progress.md` criado (log do que foi feito / erros / testes)

- [x] `gemini.md` criado com a CONSTITUTION integral (schemas, enums, invariants, allowlists)
- [x] `.env.example` criado sem valores reais
- [x] 12 SOPs criados em `architecture/` (00_overview → 90_self_annealing)
- [x] Fixtures de teste criados em `fixtures/` (match_scheduled, match_live_goal, news_bundle)
- [] **Confirmação humana** para iniciar a Phase L (“pode ir para o Link”) ← aguardando

Phase 2 — L: Link (handshakes)

Objetivo: provar cada integração com um script mínimo em `tools/` (um arquivo por handshake). Se um ping falhar: registrar em `progress.md` + `findings.md` e não avançar a tool de lógica daquela integração (as demais seguem).

- [] `tools/ping_sheets.py` — lê/escreve uma célula de teste
- [] `tools/ping_webhook.py` — posta “pong” no Slack/Discord
- [] `tools/ping_football_api.py` — GET próximo jogo do Flamengo
- [] `tools/ping_rss.py` — fetch 1 item de cada fonte allowlist
- [] `tools/ping_llm.py` — classifica 1 headline de fixture e valida o JSON Schema
- [] Pings obrigatórios 1, 2 e 4 verdes antes de implementar o pipeline completo (football API pode ficar mockada)

Phase 3 — A: Architect (3 camadas)

Objetivo: SOPs escritos ANTES das tools de lógica; tools atômicas na ordem da Navigation.

Layer 1 — architecture/ (SOPs)

- [x] `architecture/00_overview.md` — fluxo do dia
- [x] `architecture/10_ingest_news.md`
- [x] `architecture/11_ingest_match.md`
- [x] `architecture/12_ingest_social.md`
- [x] `architecture/20_classify_and_dedupe.md`
- [x] `architecture/21_write_cards.md`
- [x] `architecture/30_build_daily_payload.md`
- [x] `architecture/31_match_ticks.md`
- [x] `architecture/40_editor_queue.md`
- [x] `architecture/50_publish.md`
- [x] `architecture/60_venues.md`
- [x] `architecture/90_self_annealing.md`
- [] Cada SOP validado contra fixtures (na Phase L, junto das tools)

Layer 2 — Navigation (ordem de chamada)

```

ingest_news → ingest_match → ingest_social
    ↓
classify_and_dedupe → write_cards
    ↓
build_daily_payload → send_editor_queue
    ↓
(aguardar aprovação humana no MVP)
    ↓
publish_payload

```

Matchday (worker paralelo): `poll_match` → if event: `emit_match_tick` → `publish_tick`

Layer 3 — tools/ (MVP, ordem de construção)

- [] `ingest_news.py`
- [] `ingest_match.py`
- [] `classify_and_dedupe.py`
- [] `write_cards.py`
- [] `build_daily_payload.py`
- [] `send_editor_queue.py`
- [] `publish_payload.py`
- [] `ingest_venues.py` (lê o sheet, não scrapa Google)
- [] `emit_match_tick.py`

Fase 2 (NÃO construir agora): `ingest_social_x.py` , `rank_memes.py` , `send_whatsapp.py` , `send_email_digest.py` , `geocode_venues.py` .

Automações do dia (o que o Trigger vai rodar) — timezone America/Sao_Paulo

Trigger	Hora / evento	Tools	Aprovação?
Cron daily ingest	06:30	<code>ingest_*</code>	não
Cron daily build	07:15	<code>classify</code> , <code>write_cards</code> , <code>build</code> , <code>send_queue</code>	sim (fila)
Cron publish	07:45	<code>publish</code> se approved	—
Cron refresh	12:00 e 18:00	<code>ingest</code> + rebuild cards novos	cards novos na fila
Matchday poll	kickoff-90min até FT+30	<code>poll_match</code> a cada 30s	ticks da API sem aprovação
Venue weekly	segunda 10:00	<code>ingest_venues</code>	venues novos pending

Modo matchday se existir jogo `professional_m` nas próximas 18 horas.

Phase 4 — S: Stylize

Objetivo: bloco de aprovação e home pública do MVP. Apresentar o HTML gerado ao humano ANTES de considerar Stylize fechado.

- [] Bloco Slack/Discord “Hoje na Nação” (🔴 AGORA / 🏠 NAÇÃO / 🎬 CORTES / 📍 PERTO DE VOCÊ + botões ✅ [Publicar] ✎ [Pedir edição] 🗑️ [Segurar])
- [] Home HTML MVP: fundo preto/vermelho/branco; SEM escudo oficial; 4 seções do payload; mobile first; cards com selo colorido + fonte + “ler na origem”; rodapé “Veículo independente. Não somos o Flamengo.”
- [] Home renderiza o último payload approved

Phase 5 — T: Trigger

Objetivo: automação no ar e maintenance log preenchido.

- [] Confirmar payload publicado na cloud (sheet `daily_payload` + HTML no ar)
- [] Agendar crons (GitHub Actions ou Cloud Scheduler; default: GitHub Actions, cron em BRT)
- [] Maintenance Log em `gemini.md`: última execução ok, rate limits, fontes que falharam nas últimas 24h, editor(a) de plantão

Definition of Done do MVP

- [] Constitution em `gemini.md`
- [] Pings 1, 2, 4 verdes
- [] Pipeline daily gera Daily Payload válido no schema
- [] Fila no Slack/Discord com 3 botões conceituais (mesmo que o callback seja manual no início)
- [] Home HTML renderiza o último payload approved
- [] 5 cidades no sheet venues (podem ter 0 itens)
- [] Cron 07:15 documentado
- [] Nenhum vídeo oficial do clube no módulo cortes
- [] Rodapé “independente” visível

Restrições de execução (valem para todas as fases)

- Não escrever o site inteiro no dia 1.
- Não escrever `ingest_social_x` no dia 1.
- Não inventar 200 bares: mapa começa com as 5 cidades e planilha vazia + 1 formulário de cadastro documentado no SOP `60_venues.md`.
- Toda tool de lógica precisa passar nos fixtures de `fixtures/` antes de tocar em API real.